

LABORATORIO DI SISTEMI OPERATIVI

Semafori

fabio.rossi@unipg.it

075 5855005

Sincronizzazione

- Sincronizzazione di Processi o *Threads* per eseguire coerentemente una serie di azioni, per esempio per accedere ordinatamente a sezioni critiche di un programma o a risorse condivise
- Sincronizzazione dell'accesso ai dati per garantirne integrità e coerenza

Semafori

- Primitive di sincronizzazione
- Due implementazioni: *POSIX* e *SYSTEM V*
- *POSIX*: più semplice ma meno diffusa, specialmente in sistemi vecchi
- *SYSTEM V*: API "tradizionale"
- **Nel seguito vedremo solo i *POSIX***

ndr: per informazioni sui meccanismi di IPC System V vedi man 7 sysvipc

Semafori

- ***man 7 sem_overview*** (per i *POSIX*)
- Rappresentabili come variabili intere
- Mai minori di 0
- Due operazioni possibili:
 - *POST*: incrementa di 1
 - *WAIT*: decrementa di 1

Semafori

- Non potendo essere negativi, se una *WAIT* trova la variabile uguale a 0 si blocca fin quando il valore non diviene positivo. Solo allora il decremento può essere effettuato e la *WAIT* ritorna

Semafori POSIX

- ***Named***
- ***Unnamed***
- P.S.: può essere necessario linkare col flag `-pthread`

Semafori POSIX Named

- Sono identificati da un nome nella forma «/somename»
- ***sem_open(3)***: apre o crea (se non esiste) un semaforo *named*
- Due processi/*thread* possono operare sullo stesso semaforo passando lo stesso nome a ***sem_open(3)***
- Dopo averlo aperto i processi possono operare con ***sem_post(3)*** e ***sem_wait(3)***

Semafori POSIX Named

- Quando un semaforo non serve più il processo può chiudere il suo accesso con ***sem_close(3)***
- Quando tutti i processi hanno terminato di usarlo il semaforo può essere rimosso con ***sem_unlink(3)***
- **Sono persistenti nel kernel, cioè se non rimossi permangono fino al riavvio della macchina**

Semafori POSIX Named

```
#include <fcntl.h>           /* For O_* constants */
#include <sys/stat.h>        /* For mode constants */
#include <semaphore.h>

sem_t *sem_open(const char *name, int oflag);
sem_t *sem_open(const char *name, int oflag,
                mode_t mode, unsigned int value);
```

- apre un semaforo *named* (o lo crea se non esiste e si specifica il flag **O_CREAT**)
- Gli **oflag** sono analoghi a quelli per la *open(2)*
- **mode** specifica i permessi, come in *open(2)*
- **value** è il valore iniziale per il semaforo
- Se **O_CREAT** viene specificato allora **mode** e **value** devono essere forniti (ma se il semaforo già esiste sono ignorati)
- l'utente proprietario del semaforo eventualmente creato è lo stesso utente proprietario del processo chiamante

Ritorno: torna l'indirizzo del semaforo. In caso di errore torna **SEM_FAILED** e setta **errno**

Semafori POSIX Named: sequenza di utilizzo

```
...
#include <fcntl.h>           /* For O_* constants */
#include <sys/stat.h>        /* For mode constants */
#include <semaphore.h>

...
sem_t *sem;
...
sem = sem_open("/mysem", O_CREAT, S_IRWXU, 0);
...
sem_post(sem);    o sem_wait(sem);    (o sem_timedwait())
...
sem_close(sem);
...
sem_unlink("/mysem");
```

Semafori POSIX Named

- In Linux sono «visibili» ed accessibili dal filesystem, generalmente in ***/dev/shm***
- Hanno nomi nella forma ***sem.somename***
- Possiamo assegnare permessi

Semafori POSIX Unnamed

- Non hanno nome
- Allocati in un'area di memoria condivisa tra i processi/*thread* che li usano
- *Thread-shared*: l'area di memoria può essere una variabile globale del processo (ovviamente per *thread* di uno stesso processo...)
- *Process-shared*: allocati in una ***shared memory***
 - *SYSTEM V*: creata con ***shmget(2)***
 - *POSIX*: creata con ***shm_open(3)***

Semafori POSIX Unnamed

- Inizializzati con ***sem_init(3)***
- Dopo l'inizializzazione si può operare con ***sem_post(3)*** e ***sem_wait(3)/sem_timedwait(3)***
- Quando non più necessari, prima di deallocare un'eventuale area di memoria allocata per essi, dovrebbero essere distrutti con ***sem_destroy(3)***, altrimenti in certe implementazioni certe risorse potrebbero rimanere occupate (*leak*)

Semafori POSIX Unnamed

```
#include <semaphore.h>
```

```
int sem_init(sem_t *sem, int pshared, unsigned int value);
```

- inizializza un semaforo *unnamed* all'indirizzo specificato da **sem**
- **pshared** può essere uguale a:
 - 0: semaforo condiviso tra threads di uno stesso processo (può essere allocato in una variabile globale)
 - non 0: semaforo condiviso tra processi. Allocato in una *shared memory* (*shm_open(3)*, *mmap(2)*, and *shmget(2)*)
- **value** è il valore iniziale per il semaforo
- avremo un comportamento "indefinito" se inizializziamo un semaforo già inizializzato

Ritorno: 0 per successo. -1 per errore, **errno** settata

Semafori POSIX Unnamed: sequenza di utilizzo

```
...  
#include <semaphore.h>  
...  
sem_t sem;  
...  
sem_init(&sem, 0, 0);  
...  
sem_post(&sem);    o sem_wait(&sem); (o sem_timedwait())  
...  
sem_destroy(&sem);
```

Semafori POSIX: decremento (operazione di *wait*)

```
#include <semaphore.h>
```

```
int sem_wait(sem_t *sem);
```

- decrementa un semaforo (sia esso *unnamed* o *named*)
- se il semaforo ha il valore zero si blocca in attesa che il valore diventi positivo. Altrimenti decrementa e ritorna
- la funzione torna anche se, pur essendo il semaforo a 0, un *signal handler* la interrompe (cioè il processo riceve un signal gestito). In questo caso torna -1 ed ***errno*** è settata a ***EINTR***

```
int sem_timedwait(sem_t *sem,  
                  const struct timespec *abs_timeout);
```

- come ***sem_wait()*** ma, nel caso il semaforo sia a 0 e la funzione si blocchi, l'attesa non si protrae oltre il tempo specificato da ***abs_timeout***
- se ***sem_timedwait()*** torna perché il *timeout* è spirato prima che il semaforo diventasse positivo allora ritorna -1 ed ***errno*** viene settata a ***ETIMEDOUT***
- ***abs_timeout*** punta ad una struttura che specifica il numero di secondi dal 01/01/1970 trascorsi i quali il *timeout* spira e la chiamata comunque ritorna (vedi slide successiva)
- che lo sappia in *macOS* ***sem_timedwait()*** non è implementata!

Ritorno: 0 per successo. -1 per errore, ***errno*** settata

Semafori POSIX: utilizzo di *sem_timedwait(3)*

```
#include <semaphore.h>
int sem_timedwait(sem_t *sem,
                  const struct timespec *abs_timeout);
```

struct timespec è definita come segue (*man 3 timespec*):

```
struct timespec {
    time_t tv_sec; /* Seconds */
    long   tv_nsec; /* Nanoseconds [0 .. 999999999] */
};
```

Esempio di utilizzo di *set_timedwait(3)* con *clock_gettime(2)*, in cui è specificato il numero di secondi dal 01/01/1970 trascorsi i quali la chiamata deve comunque ritornare. :

```
struct timespec ts; sem_t sem;
.....
clock_gettime(CLOCK_REALTIME, &ts); /* imposta in ts il
    numero di secondi trascorsi dal 1-1-70 al momento della
    chiamata */
ts.tv_sec += 8; // imposta un timeout di 8 secondi
sem_timedwait(&sem, &ts); /* decrementa il semaforo ma non
    attende più di 8 secondi */
```

Esempio in *man sem_wait(3)* parte 1

```
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
#include <semaphore.h>
#include <time.h>
#include <assert.h>
#include <errno.h>
#include <signal.h>

sem_t sem;

#define handle_error(msg) \
    do { perror(msg); exit(EXIT_FAILURE); } while (0)

static void
handler(int sig)
{
    write(STDOUT_FILENO, "sem_post() from handler\n", 24);
    if (sem_post(&sem) == -1) {
        write(STDERR_FILENO, "sem_post() failed\n", 18);
        _exit(EXIT_FAILURE);
    }
}

int
main(int argc, char *argv[])
{
    struct sigaction sa;
    struct timespec ts;
    int s;

    if (argc != 3) {
        fprintf(stderr, "Usage: %s <alarm-secs> <wait-secs>\n",
            argv[0]);
        exit(EXIT_FAILURE);
    }

    if (sem_init(&sem, 0, 0) == -1)
        handle_error("sem_init");
```

Esempio in *man sem_wait(3)* parte 2

```
/* Establish SIGALRM handler; set alarm timer using argv[1] */

sa.sa_handler = handler;
sigemptyset(&sa.sa_mask);
sa.sa_flags = 0;
if (sigaction(SIGALRM, &sa, NULL) == -1)
    handle_error("sigaction");

alarm(atoi(argv[1]));

/* Calculate relative interval as current time plus
   number of seconds given argv[2] */

if (clock_gettime(CLOCK_REALTIME, &ts) == -1)
    handle_error("clock_gettime");

ts.tv_sec += atoi(argv[2]);

printf("main() about to call sem_timedwait()\n");
while ((s = sem_timedwait(&sem, &ts)) == -1 && errno == EINTR)
    continue;      /* Restart if interrupted by handler */

/* Check what happened */

if (s == -1) {
    if (errno == ETIMEDOUT)
        printf("sem_timedwait() timed out\n");
    else
        perror("sem_timedwait");
} else
    printf("sem_timedwait() succeeded\n");

exit((s == 0) ? EXIT_SUCCESS : EXIT_FAILURE);
}
```